UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

CS3014 - Estructuras de Datos Avanzadas

Strings I
Semana 12

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

30 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Formular *pattern matching* y entender bordes y la función prefijo
- Buscar un patrón en $O(n + m)$ con KMP

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Formular *pattern matching* y entender bordes y la función prefijo
- Buscar un patrón en $O(n + m)$ con KMP
- Presentar el trie
- Combinar KMP y trie: Aho–Corasick busca muchos patrones a la vez

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar un texto dentro de otro

Buscadores, editores, bioinformática, antivirus: todos preguntan si una cadena P (el *patrón*) ocurre en otra, T (el *texto*), y dónde.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Buscar un texto dentro de otro

Buscadores, editores, bioinformática, antivirus: todos preguntan si una cadena P (el *patrón*) ocurre en otra, T (el *texto*), y dónde.

¿Hace falta mirar cada posición desde cero?

Si buscamos aaab en aaaaaaaaab, cada intento compara casi todo el patrón antes de fallar. ¿Qué sabemos *ya* del texto después de un fallo? Y si son **miles** de patrones, ¿hay que leer el texto una vez por cada uno?

Cadenas y pattern matching

Notación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Cadenas

Un alfabeto Σ ordenado; una cadena $S = S[0]S[1]\cdots S[n-1]$ tiene longitud $|S| = n$. La cadena vacía es ε.

Partes de una cadena

- **Subcadena** $S[i..j]$, con $0 \leq i \leq j < n$.
- **Prefijo** $S[0..i]$ y **sufijo** $S[i..n-1]$.
- Un prefijo (o sufijo) es **propio** si es distinto de S.

Comparamos cadenas en orden lexicográfico y $\text{LCP}(S, T)$ es la longitud del prefijo común más largo.

Pattern matching

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema

Dadas una cadena T de longitud n (texto) y una cadena P de longitud m (patrón), ¿ocurre P en T como subcadena? Variante usual: listar **todas** las posiciones i con $T[i..i+m-1] = P$.

Dos escenarios

P se conoce primero: procesamos P y luego llega T (hoy: KMP). **T se conoce primero:** procesamos T y luego llegan muchos P (próximas clases: suffix array, suffix tree, autómata de sufijos).

Fuerza bruta

Algoritmo 1: BÚSQUEDA-INGENUA(P, T)

para $i \leftarrow 0$ **a** $n - m$ **hacer**
    $j \leftarrow 0$
    mientras $j < m$ **y** $T[i + j] = P[j]$ **hacer** $j \leftarrow j + 1$
    si $j = m$ **entonces** reportar i

$O(nm)$ en el peor caso

Con $T = \mathrm{a}^n$ y $P = \mathrm{a}^{m-1}\mathrm{b}$, cada posición compara $m - 1$ caracteres. Se *olvida* lo ya comparado.

Bordes y función prefijo

Borde de una cadena

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **borde** de S es una cadena no vacía que es a la vez **prefijo propio** y **sufijo propio** de S.

Ejemplos

abaab: el único borde es ab (a no sirve, pues abaab termina en b). aabaa: bordes aa y a.

Un borde de longitud b significa que $S[0..b-1] = S[n-b..n-1]$: S se superpone consigo misma.

Función prefijo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

$\pi[i]$ es la longitud del **borde más largo** de $P[0..i]$ (el prefijo de longitud $i+1$), o 0 si no tiene.

Función prefijo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

$\pi[i]$ es la longitud del **borde más largo** de $P[0..i]$ (el prefijo de longitud $i+1$), o 0 si no tiene.

Ejemplo: $P =$ ababaca

i	0	1	2	3	4	5	6
$P[i]$	a	b	a	b	a	c	a
$\pi[i]$	0	0	1	2	3	0	1

Cómo calcularla

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

De $\pi[i-1]$ a $\pi[i]$

Sea $k = \pi[i-1]$: el prefijo $P[0..k-1]$ es sufijo de $P[0..i-1]$.

- Si $P[i] = P[k]$, el borde se extiende: $\pi[i] = k+1$.
- Si no, se prueba el siguiente borde más corto: $k \leftarrow \pi[k-1]$, y se repite.
- Si k llega a 0 y $P[i] \neq P[0]$, $\pi[i] = 0$.

En particular $\pi[i] \leq \pi[i-1] + 1$: el valor sube a lo más 1 por paso.

Función prefijo: algoritmo

Algoritmo 2: PREFIJO(P, m)

$\pi[0] \leftarrow 0; \quad k \leftarrow 0$
para $i \leftarrow 1$ **a** $m-1$ **hacer**
  mientras $k > 0$ ***y*** $P[i] \neq P[k]$ **hacer** $k \leftarrow \pi[k-1]$
  si $P[i] = P[k]$ **entonces** $k \leftarrow k+1$
  $\pi[i] \leftarrow k$
devolver π

$O(m)$ (análisis agregado)

k sube a lo más 1 por iteración del **for**, así que sube $\leq m-1$ veces en total. Cada vuelta del **while** lo baja al menos 1 y $k \geq 0$: hay a lo más $m-1$ vueltas del **while** en total.

Una traza

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$P =$ ababaca

i	$P[i]$	k antes $\rightarrow$ después	qué pasa
1	b	$0 \rightarrow 0$	$P[1] \neq P[0]$
2	a	$0 \rightarrow 1$	coincide con $P[0]$
3	b	$1 \rightarrow 2$	coincide con $P[1]$
4	a	$2 \rightarrow 3$	coincide con $P[2]$
5	c	$3 \rightarrow 1 \rightarrow 0$	falla; baja $\pi[2] = 1$, luego $\pi[0] = 0$
6	a	$0 \rightarrow 1$	coincide con $P[0]$

Knuth–Morris–Pratt

No retroceder en el texto

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

La idea

Si leímos $T[i-k..i-1] = P[0..k-1]$ y $T[i] \neq P[k]$, el siguiente intento **no** debe empezar en $T[i-k+1]$ con $j = 0$: ya sabemos que los últimos caracteres leídos son un **borde** de $P[0..k-1]$.

Entonces

Se retrocede en el *patrón*, no en el texto: $k \leftarrow \pi[k-1]$. El puntero al texto i **solo avanza**.

KMP

Algoritmo 3: $\mathrm{KMP}(P, T)$

$\pi \leftarrow \textsc{Prefijo}(P, m)$; $\quad k \leftarrow 0$
para $i \leftarrow 0$ **a** $n-1$ **hacer**
  mientras $k > 0$ ***y*** $(k = m$ ***o*** $T[i] \neq P[k])$ **hacer** $k \leftarrow \pi[k-1]$
  si $T[i] = P[k]$ **entonces** $k \leftarrow k+1$
  si $k = m$ **entonces** reportar $i - m + 1$

KMP

Algoritmo 4: $\mathrm{KMP}(P, T)$

$\pi \leftarrow \textsc{Prefijo}(P, m)$; $\quad k \leftarrow 0$
para $i \leftarrow 0$ **a** $n - 1$ **hacer**
  mientras $k > 0$ ***y*** $(k = m$ ***o*** $T[i] \neq P[k])$ **hacer** $k \leftarrow \pi[k-1]$
  si $T[i] = P[k]$ **entonces** $k \leftarrow k + 1$
  si $k = m$ **entonces** reportar $i - m + 1$

$O(n + m)$

Mismo argumento: k sube a lo más n veces y cada vuelta del **while** lo baja. Con el preprocesamiento de P son $O(n + m)$ comparaciones.

Una traza de KMP

$P = \texttt{aba}$, $\pi = [0, 0, 1]$, $T = \texttt{abababa}$

i	0	1	2	3	4	5	6
$T[i]$	a	b	a	b	a	b	a
k tras leer	1	2	3	2	3	2	3
ocurrencia en			0		2		4

En $i = 2$, $k = m = 3$ y se reporta 0; para seguir se usa $k \leftarrow \pi[2] = 1$ (el borde a), y las ocurrencias que se **superponen** se encuentran igual.

Aplicaciones de π

Periodos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Periodo

p es un **periodo** de S si $S[i] = S[i+p]$ para todo $0 \leq i < n - p$. El periodo mínimo es $p_{\text{mín}} = n - \pi[n-1]$.

Periodos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Periodo

p es un **periodo** de S si $S[i] = S[i+p]$ para todo $0 \leq i < n - p$. El periodo mínimo es $p_{\text{mín}} = n - \pi[n-1]$.

Por qué

Un periodo p equivale a un borde de longitud $n - p$. El periodo **menor** es el del borde **mayor**: $n - \pi[n-1]$.

S es una potencia de una cadena más corta si y solo si $p_{\text{mín}}$ divide a n y $p_{\text{mín}} < n$.

Ocurrencias de cada prefijo

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Cuántas veces aparece cada prefijo de S en S?

Sea $cnt[L]$ el número de ocurrencias del prefijo de longitud L. Cada ocurrencia del prefijo de longitud L **contiene** una ocurrencia de su borde máximo.

Algoritmo 5: OCURRENCIAS-PREFIJOS(π, n)

$cnt[L] \leftarrow 1$ para $L = 1, \ldots, n$
para $L \leftarrow n$ ***hasta*** 1 **hacer**
  si $\pi[L-1] > 0$ **entonces** $cnt[\pi[L-1]] \leftarrow cnt[\pi[L-1]] + cnt[L]$

Trie

Trie

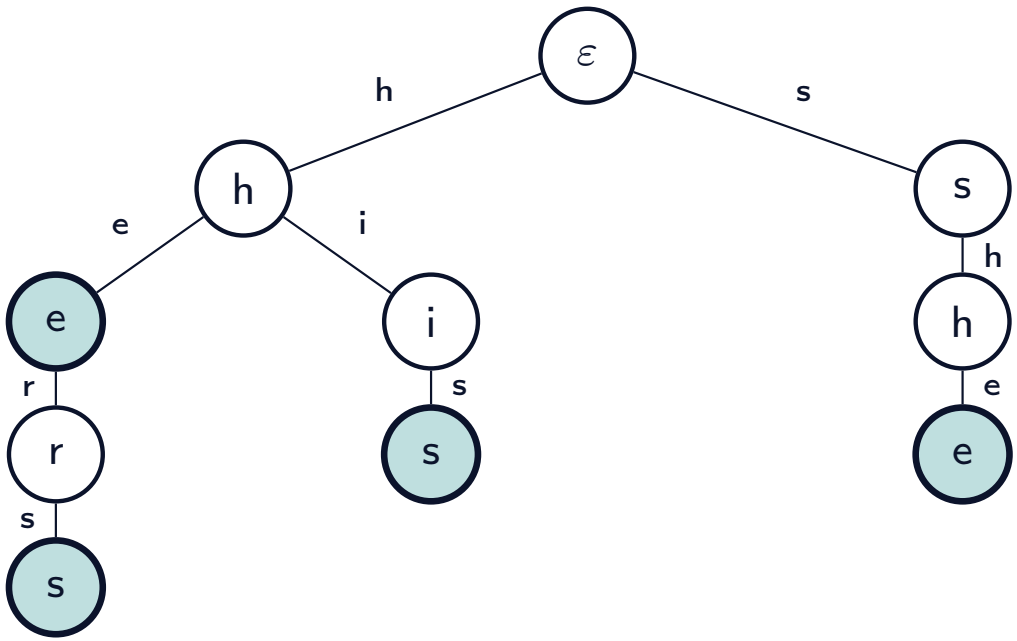
UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Un **trie** (árbol de prefijos) de un conjunto de cadenas es un árbol con raíz en el que

- cada arista está etiquetada con un carácter y los hijos de un nodo tienen etiquetas distintas;
- cada nodo representa el prefijo formado por las etiquetas del camino desde la raíz;
- un nodo es **terminal** si su prefijo es una cadena del conjunto.

Ejemplo: {he, she, his, hers}



10 nodos (con la raíz) para 12 caracteres: los prefijos comunes (h) se comparten.
Los nodos terminales van en verde.

Insertar y buscar

Algoritmo 6: INSERTAR(S)

$v \leftarrow$ raíz
para $c \in S$ **hacer**
    si *v no tiene hijo con etiqueta c* **entonces** crear el hijo
    $v \leftarrow$ hijo de v con etiqueta c
marcar v como terminal

Costo

Insertar y buscar cuestan $O(|S|)$, **sin depender** de cuántas cadenas haya. El número de nodos es a lo más $1 + \sum |S_i|$.

Estado = prefijo más largo que sirve

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Invariante

Después de leer $T[0..i]$, el autómata está en el nodo cuyo prefijo es el **sufijo más largo** de $T[0..i]$ que es prefijo de *algún* patrón.

Equivalente a KMP

Con un solo patrón, el trie es una cadena y el estado es exactamente k, la longitud del borde actual. Con varios, el estado es un nodo.

Enlace de falla

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Definición

Para un nodo v con prefijo $\text{str}(v) \neq \varepsilon$, $\text{fail}(v)$ es el nodo del **sufijo propio más largo** de $\text{str}(v)$ que también es nodo del trie (es decir, prefijo de algún patrón). $\text{fail}(\text{raíz}) = \text{raíz}$.

Enlace de falla

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

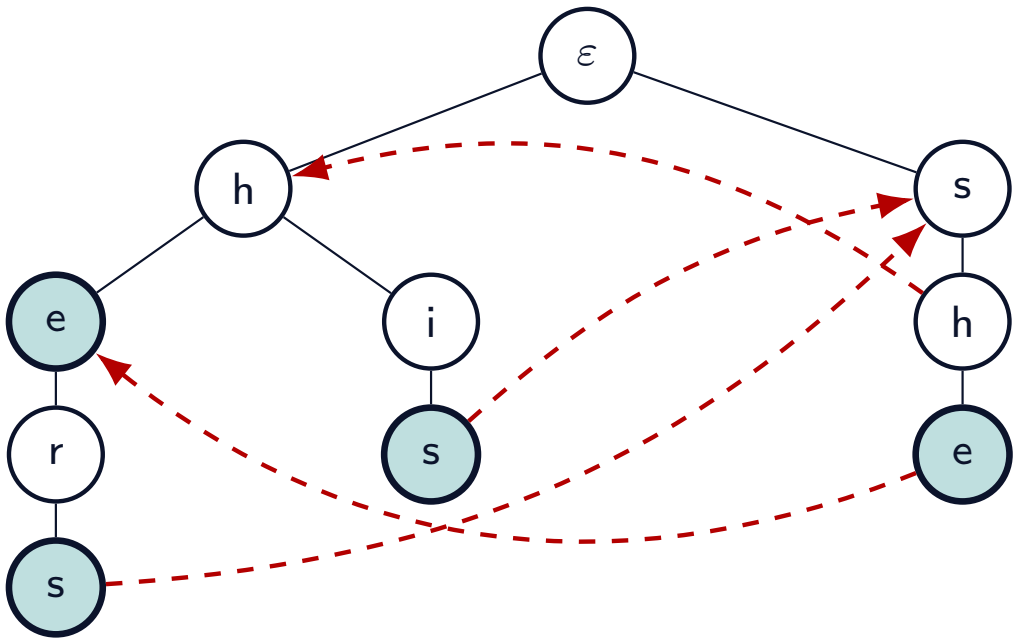
Definición

Para un nodo v con prefijo $\text{str}(v) \neq \varepsilon$, $\text{fail}(v)$ es el nodo del **sufijo propio más largo** de $\text{str}(v)$ que también es nodo del trie (es decir, prefijo de algún patrón). $\text{fail}(\text{raíz}) = \text{raíz}$.

Ejemplos

$\text{fail}(\texttt{she}) = \texttt{he}, \quad \text{fail}(\texttt{sh}) = \texttt{h}, \quad \text{fail}(\texttt{his}) = \texttt{s}, \quad \text{fail}(\texttt{hers}) = \texttt{s}.$

Los enlaces sobre el trie



Los enlaces que no se dibujan van a la raíz.

Cómo calcularlos

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Por niveles, de arriba hacia abajo

Sea $u = \delta(v, c)$ un hijo de v. Entonces

$$\text{fail}(u) = \delta\big(\text{fail}(v),\ c\big)$$

donde $\delta(w, c)$ es el nodo al que se llega desde w con c, **subiendo por fallas** hasta encontrar un nodo con hijo c.

Por qué

El sufijo propio más largo de $\text{str}(v)\,c$ que es prefijo es de la forma $s\,c$ con s un sufijo propio de $\text{str}(v)$ que es prefijo: el primero que se pueda extender con c es el de $\text{fail}(v)$, luego $\text{fail}(\text{fail}(v))$, etc.

El autómata completo

Eliminar el *while*

Definimos para todo nodo v y todo carácter c

$$\delta(v, c) = \begin{cases} \text{hijo de } v \text{ con etiqueta } c & \text{si existe} \\ \delta(\text{fail}(v), c) & \text{si no (con } \delta(\text{raíz}, c) = \text{raíz}) \end{cases}$$

Calculado por **niveles**, cada $\delta(v, \cdot)$ se obtiene de $\delta(\text{fail}(v), \cdot)$, que está en un nivel menor.

El autómata completo

Eliminar el *while*

Definimos para todo nodo v y todo carácter c

$$\delta(v, c) = \begin{cases} \text{hijo de } v \text{ con etiqueta } c & \text{si existe} \\ \delta(\text{fail}(v), c) & \text{si no (con } \delta(\text{raíz}, c) = \text{raíz}) \end{cases}$$

Calculado por **niveles**, cada $\delta(v, \cdot)$ se obtiene de $\delta(\text{fail}(v), \cdot)$, que está en un nivel menor.

Costo total: $O(N\,|\Sigma|)$ con N nodos. El procesamiento del texto queda en **una** transición por carácter.

Construcción

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 7: CONSTRUIR-AC

cola $\leftarrow$ vacía
para $c \in \Sigma$ **hacer**
  si *la raíz tiene hijo u con etiqueta c* **entonces**
    fail(u) $\leftarrow$ raíz; δ(raíz, c) $\leftarrow u$; encolar u
  en otro caso
    δ(raíz, c) $\leftarrow$ raíz
mientras *cola no vacía* **hacer**
  $v \leftarrow$ desencolar
  para $c \in \Sigma$ **hacer**
    si *v tiene hijo u con etiqueta c* **entonces**
      fail(u) $\leftarrow \delta$(fail(v), c); $\delta(v, c) \leftarrow u$; encolar u
    en otro caso
      $\delta(v, c) \leftarrow \delta$(fail($v$), c)

Enlaces de diccionario

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Qué patrones terminan aquí?

En el estado v terminan **todos** los patrones que son sufijos de str(v): los de v mismo y los de los nodos de la cadena $\text{fail}(v), \text{fail}^2(v), \ldots$

Enlace de diccionario

dic(v) es el **primer** nodo *terminal* de la cadena de fallas de v (sin contar a v), o ninguno. Se calcula al crear fail:

$$\text{dic}(v) = \begin{cases} \text{fail}(v) & \text{si es terminal} \\ \text{dic}(\text{fail}(v)) & \text{si no} \end{cases}$$

Búsqueda

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Algoritmo 8: BUSCAR-AC(T)

```
v ← raíz
para i ← 0 a n − 1 hacer
    v ← δ(v, T[i])
    u ← (v si es terminal, si no dic(v))
    mientras u existe hacer
        reportar los patrones de u que terminan en i
        u ← dic(u)
```

Búsqueda

Algoritmo 9: BUSCAR-AC(T)

```
v ← raíz
para i ← 0 a n − 1 hacer
    v ← δ(v, T[i])
    u ← (v si es terminal, si no dic(v))
    mientras u existe hacer
        reportar los patrones de u que terminan en i
        u ← dic(u)
```

$O(n + z)$

Una transición por carácter y un paso por **cada ocurrencia** reportada (z en total). No depende del número de patrones.

Una traza: $T = \texttt{ushers}$

i	0	1	2	3	4	5
$T[i]$	u	s	h	e	r	s
estado	ε	s	sh	she	her	hers
se reporta				she, he		hers

En `she` terminan `she` y, por el enlace de falla $\text{fail}(\texttt{she}) = \texttt{he}$, también `he`. En `her` se continúa desde `he` con la `r`: el estado `her` sale de $\delta(\texttt{she}, \texttt{r}) = \delta(\texttt{he}, \texttt{r})$.

Contar ocurrencias de cada patrón

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Visitas y propagación

1. Recorrer T y sumar 1 a $vis[v]$ en cada estado visitado.
2. Recorrer los nodos en orden **inverso** al de la cola (de más profundo a menos): $vis[\text{fail}(v)] \mathrel{+}= vis[v]$.
3. El número de ocurrencias del patrón P es vis de su nodo.

$O(n + N)$

Sin importar cuántas ocurrencias haya: se cuentan sin enumerarlas.

Aho–Corasick = KMP + trie

	KMP	Aho–Corasick
Patrones	uno	muchos
Estado	longitud del borde k	nodo del trie
Retroceso	$\pi[k-1]$	fail(v)
Reporte	$k = m$	enlaces de diccionario
Costo	$O(n+m)$	$O(n + \sum \lvert P \rvert \lvert \Sigma \rvert + z)$

Es el mismo algoritmo

Con un solo patrón, el trie es una cadena, fail es π y δ es el autómata de KMP.

Práctica

Práctica

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Problema	**Idea**
CSES 1753 — String Matching	KMP directo
CSES 1732 — Finding Borders	cadena de bordes con π
CSES 1733 — Finding Periods	periodos $= n-$bordes
Codeforces 126B — Password	prefijo que es borde y aparece en el medio
CSES 2102 — Finding Patterns	Aho–Corasick
CSES 2103 — Counting Patterns	conteo con el árbol de fallas
HDU 2222 — Keywords Search	Aho–Corasick básico

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un borde es un prefijo propio que también es sufijo; KMP no retrocede en el texto y baja por la cadena de bordes: $O(n + m)$.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un borde es un prefijo propio que también es sufijo; KMP no retrocede en el texto y baja por la cadena de bordes: $O(n + m)$.
- Aplicaciones de π: periodos, bordes y ocurrencias de prefijos.
- Un trie comparte prefijos y es la base del autómata.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un borde es un prefijo propio que también es sufijo; KMP no retrocede en el texto y baja por la cadena de bordes: $O(n + m)$.
- Aplicaciones de π: periodos, bordes y ocurrencias de prefijos.
- Un trie comparte prefijos y es la base del autómata.
- Aho–Corasick: el estado es el sufijo más largo del texto que es prefijo de un patrón; fallas por niveles, diccionario para reportar: $O(n + z)$.

Logros de la sesión



Logros de la sesión

- Sabemos construir la función prefijo y justificar su costo
- Sabemos buscar un patrón con KMP y calcular periodos

Logros de la sesión

- Sabemos construir la función prefijo y justificar su costo
- Sabemos buscar un patrón con KMP y calcular periodos
- Sabemos construir un trie
- Sabemos buscar muchos patrones y contar ocurrencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias

Referencias

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Bibliografía citada

- Knuth, D. E., Morris, J. H., & Pratt, V. R. (1977). Fast pattern matching in strings. *SIAM J. Comput.*, 6(2), 323–350.
- Aho, A. V., & Corasick, M. J. (1975). Efficient string matching: an aid to bibliographic search. *Commun. ACM*, 18(6), 333–340.
- Gusfield, D. (1997). *Algorithms on Strings, Trees, and Sequences*. Cambridge University Press.

Agradecimientos

Estas diapositivas están basadas en las clases de Luciano A. Romero Calla para este curso (a su vez basadas en las clases de Erik Demaine, MIT 6.851), así como en clases similares de otras universidades.
Lista de créditos: Luciano A. Romero Calla, Erik Demaine, Keith Schwarz.